Lexique

Les termes techniques du cours, en une phrase, avec leur équivalent anglais

Kevin Bardoux

Table des matières

A 1
B 2
C 2
D 3
E 3
F 4
G 4
H 4
I 4
L 4
M 5
N 5
P 5
R 6
S 6
T 6
V 7

Chaque terme est défini en une phrase, avec son équivalent anglais entre parenthèses et la semaine où il apparaît. Le lexique grandit au fil de la formation : un terme y entre avec sa semaine.

A

Adresse IP (*IP address*) : Numéro qui identifie un appareil sur un réseau. *Semaine 4*.

Algorithme (*algorithm*) : Suite d'instructions précises, exécutées dans l'ordre, qui résout un problème. *Semaine 1*.

Argument (*argument*) : Valeur donnée à un programme ou à une fonction au moment de l'appel. *Semaine 2*.

B

Binaire (*binary*) : Façon d'écrire les nombres avec deux chiffres seulement, `0` et 1. *Semaine 2*.

Bit (*bit*) : Chiffre binaire : `0` ou 1, la plus petite information que manipule un ordinateur. *Semaine 2*.

Bogue (*bug*) : Erreur dans un programme, qui le fait planter ou donner un résultat faux. *Semaine 1*.

Booléen (*boolean*) : Valeur qui ne peut être que `true` (vrai) ou `false` (faux). *Semaine 2*.

Boucle (*loop*) : Instruction qui répète un groupe d'instructions tant qu'une condition est vraie ou un nombre de fois donné. *Semaine 1*.

Branche (*branch*) : Ligne de travail à part dans un dépôt, qui ne touche pas à `main`. *Semaine 4*.

C

Chaîne de caractères (*string*) : Valeur de type texte, écrite entre guillemets. *Semaine 2*.

Clé SSH (*SSH key*) : Paire de clés : la clé publique se partage, la clé privée ne quitte jamais votre poste. *Semaine 4*.

Client (*client*) : Programme (le navigateur) qui demande une ressource à un serveur. *Semaine 4*.

Code de statut (*status code*) : Nombre de la réponse HTTP qui dit si tout s'est bien passé (`200`) ou non (`404`, `500`). *Semaine 4*.

Commentaire (*comment*) : Texte du code ignoré par la machine, écrit pour les humains (après `//`). *Semaine 2*.

Commit (*commit*) : Étape enregistrée de l'historique de Git, avec un message. *Semaine 4*.

Condition (*condition*) : Question dont la réponse est vraie ou fausse, qui décide si des instructions s'exécutent. *Semaine 1*.

Conflit (de fusion) (*merge conflict*) : Situation où deux branches modifient la même ligne : Git demande de choisir. *Semaine 4*.

Constante (*constant*) : Variable déclarée avec `const` : son nom désigne toujours la même valeur. *Semaine 2*.

D

Débogage (*debugging*) : Démarche qui trouve et corrige la cause d'un comportement faux : lire, isoler, tester, corriger. *Semaine 3*.

Dépôt (*repository*) : Dossier suivi par Git, avec tout son historique. *Semaine 4*.

Disque (*disk, drive*) : Support qui garde les fichiers quand l'ordinateur est éteint. *Semaine 2*.

DNS (*Domain Name System*) : Service qui traduit un nom de domaine en adresse IP. *Semaine 4*.

Dossier courant (*working directory*) : Dossier dans lequel le terminal exécute les commandes ; `pwd` l'affiche. *Semaine 4*.

E

ECF (—) : Évaluation en cours de formation : une épreuve qui mesure des compétences du titre. *Semaine 3*.

Éditeur de code (*code editor*) : Logiciel pour écrire et enregistrer des fichiers de programmation en texte brut (VS Code dans la formation). *Semaine 2*.

Égalité stricte (*strict equality*) : Comparaison `===` : vraie si les deux valeurs ont le même type et la même valeur. *Semaine 2*.

ESLint (*ESLint*) : Outil qui relit le code sans le lancer et signale chaque écart à une règle. *Semaine 2*.

Événement (*event*) : Ce qui se produit (un clic, une touche, le drapeau vert) et qui déclenche un script. *Semaine 1*.

Extension de fichier (*file extension*) : Fin du nom d'un fichier, après le point, qui indique son type (`.js`, `.txt`). *Semaine 2*.

F

Fonction (*function*) : Bloc d'instructions nommé, avec des paramètres, qui renvoie un résultat quand on l'appelle. *Semaine 3*.

G

Git (*Git*) : Gestionnaire de versions : il enregistre l'historique d'un projet, étape par étape. *Semaine 4*.

Git Bash (*Git Bash*) : Terminal installé avec Git sous Windows, qui accepte les mêmes commandes que macOS et Linux. *Semaine 4*.

GitHub (*GitHub*) : Service en ligne qui héberge des dépôts Git et permet de travailler à plusieurs. *Semaine 4*.

`.gitignore` (*gitignore*) : Fichier qui liste ce que Git ne doit pas suivre. *Semaine 4*.

Grand O (*Big O*) : Notation qui dit comment le nombre d'étapes d'un algorithme grandit quand les données grandissent. *Semaine 3*.

H

HTTP (*HyperText Transfer Protocol*) : Protocole des échanges entre client et serveur : une requête, puis une réponse. *Semaine 4*.

HTTPS (*HTTP Secure*) : Version chiffrée de HTTP. *Semaine 4*.

I

IA tutrice (*AI tutor*) : Assistant d'intelligence artificielle qui explique et guide, sans faire le travail (palier 1). *Semaine 4*.

Index (Git) (*staging area*) : Zone où l'on prépare, avec `git add`, ce que contiendra le prochain commit. *Semaine 4*.

Index (tableau) (*index*) : Numéro de position d'une valeur dans un tableau ou d'un caractère dans une chaîne, à partir de `0`. *Semaine 3*.

L

Littéral de gabarit (*template literal*) : Texte écrit entre accents graves, dans lequel `${…}` insère une valeur. *Semaine 2*.

LTS (*Long Term Support*) : Version d'un logiciel maintenue longtemps, recommandée pour un usage sérieux. *Semaine 2*.

M

Mémoire vive (*RAM*) : Mémoire qui contient les programmes en cours et leurs données ; elle s'efface quand l'ordinateur s'éteint. *Semaine 2*.

Message (Scratch) (*broadcast*) : Signal nommé envoyé par un sprite, qui démarre les scripts des sprites qui l'attendent. *Semaine 2*.

Méthode (*method*) : Fonction attachée à une valeur, appelée avec un point : `texte.toUpperCase()`. *Semaine 3*.

N

NaN (*Not a Number*) : Valeur obtenue quand une conversion en nombre échoue (`Number('abc')`). *Semaine 2*.

Node.js (*Node.js*) : Programme qui exécute du JavaScript en dehors du navigateur, dans un terminal. *Semaine 2*.

npm (*npm*) : Gestionnaire de paquets de Node.js : il télécharge et installe les paquets d'un projet. *Semaine 2*.

P

Paquet (*package*) : Code écrit par d'autres, publié sur npm, qu'on installe dans un projet. *Semaine 2*.

Paramètre (*parameter*) : Nom, dans la déclaration d'une fonction, qui reçoit une valeur à chaque appel. *Semaine 3*.

Port (*port*) : Numéro qui désigne un service précis sur une machine (2220 pour Bandit). *Semaine 4*.

Portée (*scope*) : Partie du programme où une variable est visible et utilisable. *Semaine 3*.

Processeur (*processor, CPU*) : Composant qui exécute les instructions d'un programme, une à la fois. *Semaine 2*.

Pseudo-code (*pseudocode*) : Algorithme écrit en français structuré, une instruction par ligne, avant le vrai code. *Semaine 1*.

Pull Request (*pull request*) : Proposition, sur GitHub, de fusionner une branche, après relecture. *Semaine 4*.

R

Recherche dichotomique (*binary search*) : Recherche dans une liste triée qui regarde le milieu et ne garde qu'une moitié, à chaque étape. *Semaine 3*.

Recherche linéaire (*linear search*) : Recherche qui regarde les cases une par une, du début à la fin. *Semaine 3*.

Renvoyer (`return`) (*return*) : Instruction qui termine une fonction et indique la valeur qu'elle renvoie. *Semaine 3*.

REPL (*REPL*) : Console interactive de Node : on tape une ligne, Node l'exécute et affiche le résultat. *Semaine 2*.

Requête (*request*) : Message envoyé par le client : une méthode, un chemin, des en-têtes. *Semaine 4*.

Reste (opérateur) (*remainder, modulo*) : Opérateur `%` : le reste de la division entière de deux nombres. *Semaine 2*.

S

Serveur (*server*) : Ordinateur ou programme qui répond aux demandes des clients. *Semaine 4*.

Sprite (*sprite*) : Personnage ou objet animé par un programme Scratch. *Semaine 1*.

SSH (*Secure Shell*) : Protocole qui ouvre un terminal chiffré sur une machine distante. *Semaine 4*.

Style défensif (*defensive programming*) : Façon de coder qui traite d'abord les entrées invalides, avant le cas normal. *Semaine 2*.

T

Tableau (*array*) : Liste ordonnée de valeurs, repérées par un index qui commence à `0`. *Semaine 3*.

TCP/IP (*TCP/IP*) : Protocoles qui transportent les données sur le réseau : IP achemine les paquets, TCP vérifie qu'ils arrivent dans l'ordre. *Semaine 4*.

Terminal (*terminal, shell*) : Programme où l'on tape des commandes, qu'il exécute dans un dossier. *Semaine 2*.

Test (*test*) : Petit programme qui lance le vôtre et compare le résultat obtenu au résultat attendu. *Semaine 2*.

Tri (*sort*) : Rangement des valeurs d'une liste dans l'ordre (par sélection, à bulles…). *Semaine 3*.

Type (*type*) : Sorte d'une valeur : nombre, texte, vrai ou faux… *Semaine 2*.

V

Variable (*variable*) : Nom qui désigne une valeur, pour la retrouver et la modifier plus loin dans le programme. *Semaine 1*.